

FP2: PYTHON

TEMA 1. Excepciones

Excepciones

En programación, una excepción es un objeto que representa un error o una situación anómala. Cuando ocurre, el flujo normal del programa se interrumpe y se "lanza" esa excepción. Si el programa no lo maneja, normalmente termina con un error.

CONTROL DE EXCEPCIONES

Las excepciones son muy comunes y los programas deben estar preparados para manejarlas. Se podrían manejar con bloques `if-else`, pero resulta más práctico trabajar en Python con bloques `try/except`.

Si una parte del código es propensa a lanzar una excepción, podemos encerrarla en un bloque `try`. Seguido debe ir el bloque `except` donde se maneje la excepción de la mejor manera posible ocurriese.

E.g.

```
try:
    a = int(input('a: '))
    b = int(input('b: '))
    print(a//b, a/b)
```

```
except: ZeroDivisionError: # Controla el error al poner un 0 como uno de los operandos
    print('Division by Zero')
```

Si `a:2` y `b:0` → Resultado: Division by Zero.

NOTA: Cuando ocurre una excepción en el bloque `try`, se saltan las instrucciones restantes del bloque, continuando la ejecución en el bloque `except`.

CONTROL DE MÚLTIPLES EXCEPCIONES

En el ejemplo anterior solo se tuvo en cuenta 1 excepción. Sin embargo, existen muchas otras, por ello, un bloque `try` puede contener varios bloques `except` que controlen varias excepciones.

Si se lanza una excepción, esta se controlará en su respectivo bloque `except`. Si no existiese el programa terminará al no poder gestionarla.

CONTROL DE EXCEPCIONES NO ESPECÍFICAS

El tipo de excepción puede ser omitido en el bloque `except`. En ese caso todas las excepciones que ocurra serán enviadas a este bloque.

Aunque, lo más común es contemplar primero algunas excepciones específicas y poner un bloque `except` al final para tratar aquellas que no se han tenido en cuenta.

CONTROL DE EXCEPCIONES NO ESPECÍFICAS

Además, si tenemos varias excepciones específicas distintas y se quieren controlar de la misma manera, se pueden listar en el mismo bloque except, formando una tupla.

E.g.

↳ Except No Específico

except:

```
print('algo salió mal')
```

NOTA: Controla las excepciones de la misma manera (a menos que se haya definido un bloque except que lo controle).

↳ Except con Tupla

```
except (AssertionError, ValueError):
```

```
print('Valor introducido no válido')
```

NOTA: Controla de igual manera las excepciones dentro de la tupla.

INFORMACIÓN ASOCIADA A LAS EXCEPCIONES

Como ya se ha visto, las excepciones son objetos que representan errores. Por esto, contienen información acerca del error. Se puede acceder a esta creando una variable para hacer referencia a la excepción

E.g.

try:

```
a = int(input('a: '))
```

```
b = int(input('b: '))
```

```
print(a//b)
```

except Exception as err:

```
print('Handling an exception:', err) # Se muestra como string
```

```
print(type(err)) # Conocer el tipo
```

```
print(err.args) # Más info como tupla.
```

- NOTAS**
- La variable err es una referencia a la excepción
 - Exception es un tipo de dato que representa una excepción genérica de la que derivan todos los otros tipos.

BLOQUES ADICIONALES

Detrás de los bloques except se puede incluir un bloque **else**. El código en este bloque es un buen lugar para poner el código que no necesite protección del try.

También existe un bloque **finally**, que se usa para poner el código que debe ejecutarse siempre, independientemente de si se lanzó o no una excepción. Usado para liberar recursos.

EXCEPCIONES EN FUNCIONES ESTÁNDAR

Muchas funciones y métodos estándar pueden producir excepciones de diversos tipos y se han de tener en cuenta. E.g. `Index()`. Se puede controlar usando bloques try/except

CÓMO GENERAR EXCEPCIONES?

En nuestras propias funciones podemos lanzar excepciones si fuese necesario. Para ello se dispone de la sentencia **raise**, que permite lanzar excepciones si ocurre una situación anómala. Puede haber varias sentencias raise. Las situaciones anómalas deberían de ser controladas con bloques try/except

E.g.

```
import math
```

```
def area(radius):
```

```
    if radius < 0:
```

```
        raise ValueError('radius must be positive') # lanza excepción con un mensaje descriptivo
```

1) SENTENCIA RAISE SIN ARGUMENTOS
La sentencia raise se puede usar sola, sin argumentos. Es decir, sin especificar ningún tipo de excepción.

CÓMO
GUARDAR
EXCEPCIONES

Esto se usa para **propagar una excepción** recibida hacia fuera del código. Cuando un bloque except recibe una excepción puede:

- Controlarla de manera adecuada
- Propagar hacia el código llamador

Si nadie controla la excepción, el programa termina.



- Ejecución normal
- Propagación de excepciones

Un assert es una sentencia que establece una condición que debe cumplirse para que el programa continúe su ejecución. Si la condición es verdadera, la ejecución continúa, si no, se lanza una excepción. En Python se usa **assert**

ASSERTOS

def average(items):

assert len(items) != 0, "cannot calculate the average of an empty list"
return sum(items)/len(items).

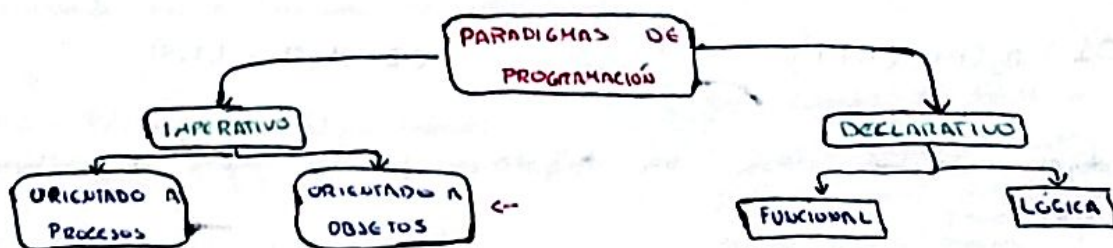
CONDICIÓN MENSAJE DE EXCEPCIÓN

Nota: Cuando el assert no se cumple se lanza una excepción: **AssertionError**.

El mensaje en el assert es opcional. Además, los asserts se ejecutan solo si la variable -- debug -- tiene valor true.

TEMA 2. CLASES

CLASES



En Programación Orientada a Objetos (POO), una 'clase' es un modelo para crear objetos de datos. Es como un puero de abstracción donde se identifica las características comunes a ciertas instancias*



NOTA: Podría compararse con el Mundo de las Ideas del filósofo griego Platón.

* En POO, instancia proviene del término inglés 'instance', del verbo 'to instance' que significa 'concretar o materializar una idea/concepto'. Son los objetos.

En Python, las clases se crean con la sentencia **class** seguida por el nombre de la clase.

DEFINICIÓN
E
INSTANCIACIÓN
DE
CLASES
I y II

E.g.

```
class MyClass:
    """ optional docstring """ #Recommendable
    pass
```

Para instanciar una clase (crear un objeto de esta), asignamos a una variable el nombre de la clase seguida de paréntesis (como llamada a una función).

E.g.

```
my-object = MyClass() # my-object es una instancia de MyClass
```

Las clases incluyen una función especial llamada **__init__** para inicializar los objetos de la clase con algunos atributos cuando se crean.

E.g.

```
class MyClass:
```

```
    def __init__(self, value1, value2):
```

```
        self.attr1 = value1
```

```
        self.attr2 = value2
```

Representa el objeto creado, a este se le añaden los atributos

atributo1 atributo2

value1 value2

se le añaden atributos al objeto.

Cuando la clase tiene la función **__init__**, en la expresión constructora se deben especificar los valores de los atributos de **__init__**, excepto el primero, pues se asocia directamente al objeto creado.

E.g.

```
O1 = MyClass(1,2)
```

```
O2 = MyClass(3,4)
```

Los objetos de una clase se caracterizan por un objeto de atributos que pueden ser:

→ Atributos de datos (variables): representan el estado del objeto

→ Funciones (métodos): implementan el comportamiento del objeto, manipulando los atributos de datos para modificar/examinar su estado.

En Python se pueden añadir atributos a un objeto escribiendo el nombre del objeto seguido de un punto, el nombre del atributo y el valor para el mismo.

E.g.

```
O1 = MyClass()
```

```
O1.size = 10
```

```
O2 = MyClass()
```

```
O2.length = 7
```

Esto es llamado **notación de punto** **<nombre objeto>.<nombre atributo>** se use tanto para crear nuevos atributos como para acceder a los ya existentes. Esto da la posibilidad de que objetos pertenecientes a una misma clase

ATRIBUTOS
DE UN
OBJETO

ATRIBUTOS
AÑADIDOS
DINÁMICAMENTE

ATRIBUTOS
AÑADIDOS
DURANTE EJECUCIÓN

tengan atributos y comportamientos diferentes, tal como se mostró antes.
No obstante, si es un modelo para la creación de objetos una clase debería declarar todo aquello que los objetos de la clase deben tener. (usando el `-init-`).

TEMA 3. Métodos

MÉTODOS DE
INSTANCIAS

Los métodos de instancia son aquellas funciones que manipulan los datos de los objetos. Estos se definen en el cuerpo de la clase del objeto.

E.g.

```
class Rectangle:
```

```
    def __init__(self, length, width):
```

```
        self.length = length
```

```
        self.width = width.
```

```
    def area(self)
```

```
        return self.length * self.width
```

2 Métodos
- `-init-`
- `area`

En Python los métodos reciben el objeto sobre el que actúan como primer parámetro formal. Esto se logra pasándole `self`.

SELF

NOTA: El nombre `self` se usa por convenio, pero Python admite que ese primer parámetro puede tener cualquier otro nombre.

Para invocar un método de un objeto se usa la notación de punto, vinculando así el método con el objeto.

E.g.

```
r1 = Rectangle(10, 5) } # instancias
```

```
r2 = Rectangle(3, 8)
```

```
print(r1.area()) # imprime 50
```

```
x = r2.area() # asigna 24 a x
```

NOTA: cuando se hace la llamada el objeto vinculado (`self`) se omite, pues se pasa implícitamente. Lo mismo ocurre cuando un método de un objeto llama a otro método del mismo.

MÉTODOS
MÁGICOS

El método `-init-` pertenece a un grupo especial de métodos de Python llamados "métodos mágicos". Estos son usados por darle a las clases de características especiales.

• SINTAXIS

Los nombres de los métodos mágicos están predefinidos y empiezan y terminan con 2 guiones bajos ("`__`"). Son llamados, generalmente, de forma implícita.*

NOTA: la sintaxis de dobles guiones bajos "`__*__`" se reserva para métodos mágicos

* Véase que al crear un objeto no se llama al método `-init-` directamente, se escribe una expresión constructora y se llama implícitamente a este método.

REPRESENTACIÓN
FORMAL
DE UN
OBJETO COMO
STRING
(--repr--)

El método mágico `__repr__` es llamado por la función predefinida `repr()`. Cuando se necesita una representación formal de un objeto como string.

E.g.

```
class Rational:
```

```
    def __init__(self, num_value, den_value):
```

```
        self.num = num_value
```

```
        self.den = den_value
```

```
    def __repr__(self):
```

```
        return f"Rational({self.num}, {self.den})" # En la medida de lo posible he de tener una  
                                                    expresion valida para crear un objeto igual.
```

```
r1 = Rational(3, 4)
```

```
print(repr(r1)) # Prints "Rational(3, 4)"
```

En el ejemplo, la función predefinida `repr` desencadena una llamada al método mágico `__repr__`. Si este último no estuviera implementado, se devolvería una string del valor por omisión similar a: `'Rational object at 0x67202a4c3'`
clase local memory en memoria

REPRESENTACIÓN
INFORMAL
DE UN
OBJETO
COMO
STRING
(.str--)

El método mágico `__str__` se llama cuando se necesita una representación informal de un objeto como string.

E.g.

```
class Rational:
```

```
    def __init__(self, num_value, den_value):
```

```
        #...
```

```
    def __str__(self):
```

```
        return self.num.__str__() + "/" + self.den.__str__()
```

```
print(r1) # Muestra "3/4"
```

La sentencia `print()` llama al método `__str__()`, en caso de que no exista llamará a `repr()`.

→ `__str__`: proporciona una representación legible del objeto.

→ `__repr__`: proporciona info para la depuración, es decir ↑ info ↓ ambigüedad.

TEMA 4. Clases como mecanismo de encapsulamiento.

El encapsulamiento es un mecanismo que facilita el agrupamiento de datos con los métodos (u otras funciones) que manipulan esos datos.

ENCAPSULAMIENTO

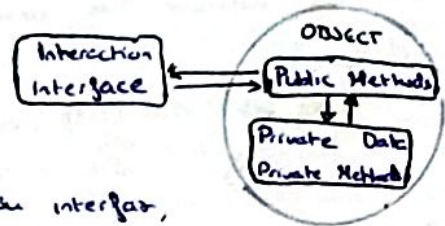
En POO, una clase hace eso mismo, por lo que es un mecanismo de encapsulamiento.

Un buen encapsulamiento implica que los datos de un objeto solo pueden ser manipulados por sus propios métodos. Para ello hay que ocultar los datos a agentes externos.

El ocultamiento de la información identifica 2 niveles principales de visibilidad: público y privado.

Ocultamiento de la información

Generalmente, los elementos privados solo son accesibles para su dueño, mientras que los públicos lo son para todo el mundo.



Los elementos públicos de una clase constituyen su interfaz, a través de la cual se interactúa con los objetos de esa clase.

Ocultar atributos en Python

- Elementos privados: Son los nombres que **empiezan con dos guiones bajos**, pero no terminan con 2 guiones bajos. No se tiene acceso desde fuera de la clase. Ej. `self.value` (público) $\Rightarrow$ `self.__value` (privado)

NOTA: Si se intenta acceder a este atributo da error.

- Elementos a tratar como privados: por convención, un solo guión bajo inicial ("_") significa 'no lo toques', trátalo como si fuese privado. No hay ningún mecanismo que impida su acceso.

NAME MANGLING

En realidad, Python no tiene un mecanismo que haga un atributo privado e impida acceder a él desde funciones que no sean métodos del objeto al que pertenece el atributo.

Lo que ocurre cuando escribimos un nombre con "_" al inicio, es que Python refuerza la privacidad añadiendo un **mangled name** $\Rightarrow$ **NombreClase__atributo**

Ej.

```
class Natural:
    def __init__(self, number):
        self.__value = number
```

```
n1 = Natural(10)
```

```
print(n1.Natural__value) # accede al atributo mediante mangled name
```

```
print(n1.value) # Error: el atributo está oculto.
```

Una vez oculto un atributo, se deben proporcionar formas de acceder a este por examinándolo o modificarlo sin saltarse el encapsulamiento. Para ello disponemos:

$\rightarrow$ **Getters** (`get-value`): nos permite observar un atributo

$\rightarrow$ **Setters** (`set-value`): permite modificar dicho atributo.

Ej.

```
class Natural:
    def __init__(self, number):
        self.__set_value(number)

    def get_value(self):
        return self.__value

    def set_value(self, new_value):
        return self.__value = new_value
```

```
n1 = Natural(10)
```

```
1 print(n1.get_value())
2 n1.set_value(12)
3 print(n1.get_value())
```

```
# 1  $\rightarrow$  10
# 3  $\rightarrow$  12
```

GETTERS Y SETTERS

• VENTAJAS

Aunque parezca que al proporcionar un getter y un setter tenemos el mismo nivel de acceso que si fuese pública, la diferencia clave es que podemos controlar los cambios que se aplican a una variable privada.

E.g.

```
def set_value(self, new_value):
```

```
    self._value = new_value if new_value >= 0 else 0 # Evita que sea negativo.
```

Otra ventaja es que al ocultar los detalles de la implementación del objeto, puede cambiarse sin que quienes lo están usando se vean afectados.

• INCONVENIENTES

Al usar getters y setters, el acceso y modificación de los datos se realiza mediante llamadas a métodos, en vez de acceder y modificar directamente las variables, lo que supone una inconveniencia sintáctica que se puede evitar usando propiedades.

Con las propiedades se puede mantener el control de acceso y modificación de los datos privados y al mismo tiempo simplificar la sintaxis.

E.g.

```
class Natural:
```

```
    def __init__(self, number):
```

```
        self._value = number
```

```
    def get_value(self):
```

```
        return self._value
```

```
    def set_value(self, new_value):
```

```
        self._value = new_value
```

```
    value = property(get_value, set_value)
```

NOTA: El property permite crear una propiedad que puede usarse como una variable pública.

• n1 = Natural(10)

print(n1.value) # Usa el setter, printa 10

n1.value = -12 # es como usar el setter

n1.value # devuelve 12

property(fget=None, fset=None, fdel=None, doc=None)

- fget: hace referencia al get, para obtener el valor de la propiedad (sin '()')
- fset: hace referencia al set, para cambiar el valor de una propiedad (sin '()')
- (si se omite tenemos una propiedad de solo lectura)
- fdel: método para eliminar un atributo (haciendo que el objeto deje de tenerlo):
- doc: string de documentación para la propiedad

• OCULTAR GETTERS Y SETTERS

Aún teniendo una propiedad, se puede seguir usando los getters y los setters, lo cual no es buena idea. Para evitar esto se puede añadir 2 guiones bajos al comienzo de estos métodos, convirtiéndolos privados. De esta manera obliga a usar la propiedad.

E.g. `__get_value(self)`

`__set_value(self, new_value)`

GETTERS
y
SETTERS

DESCRIPTOR
PROPERTY

DECORADOR
@PROPERTY

El property es útil para crear una propiedad, pero es aún más sencillo usar el decorador `@property`. Solo hay que escribir un getter con el nombre de la propiedad y decorarlo con `@property`.

E.g.

```
GETTER
@property
def value(self):
    return self._value
```

```
SETTER
@property.setter
def value(self, value_new):
    self._value = value_new
```

Necesario
añadir este
caching en el
setter

ATRIBUTOS
DE
CLASE

Los atributos de clase son aquellos compartidos por todos los objetos de la clase, en oposición a los atributos de instancia, de los cuales cada objeto tiene su versión. Los atributos de clase pueden ser tanto atributos de datos (variables de clase) como métodos de clase.

• DECLARACIÓN DE VARIABLES DE CLASE

Una variable de clase se declara en el cuerpo de la clase, fuera de los métodos.

E.g.

```
class MyClass:
    cls_attr = 0 # es un atributo de clase.

    def __init__(self, valuel, value2):
        # ... => Los atributos de clase son atributos de instancia (difícil para leer)
```

• ACCESO Y MODIFICACIÓN DE VARIABLES DE CLASE

Cualquier objeto de la clase puede acceder al valor de los atributos de su clase como si fueran suyos.

E.g.

```
o1 = MyClass(1, 2)
print(o1.cls_attr) # prints 0
```

Pero solo la clase misma puede modificarlos.

E.g.

```
MyClass.cls_attr = 2
print(o1.cls_attr) # prints 2
```

Un método de instancia puede modificar un atributo de la clase usando el atributo `__class__` del objeto al que esté vinculado (normalmente `self`).

E.g.

```
class MyClass:
    count = 0

    def __init__(self, valuel, value2):
        self.__class__.count += 1 # suma 1 al contador por objeto creado
```

Obviamente las clases también pueden acceder a sus atributos. `print(MyClass.count) # 0`

Pero si un objeto intenta cambiar el valor de una variable de clase se crea una copia de la variable, como atributo de instancia y el objeto pierde la posibilidad de acceder directamente a la variable de clase (puede hacerlo a través de la clase).

Una clase puede tener métodos que, igual que las variables de clase están asociadas a la propia clase, no a cada objeto concreto. Un método de clase se distingue porque se declara ~~con~~ el decorador

@classmethod

E.g.

```
class MyClass:
```

```
    cls.attr = 0
```

```
    @classmethod
```

```
    def sum_cls_attr(cls, value):
```

```
        cls.cls_attr += value.
```

Nota. Igual que en los métodos de instancia el primer parámetro hace referencia al objeto al que se vincula el método al ejecutarlo. En este caso el primer parámetro referencia a la clase a la que pertenece. Por convención **cls**

Los métodos de clase tienen acceso a las variables de clase y a métodos de clase, pero no puede acceder a nada relacionado con las instancias (variables o métodos), pues están asociadas a una clase.

• LLAMAR A LOS MÉTODOS DE CLASE

Los métodos de clase se invocan generalmente como:

```
MyClass.sum_cls_attr(3) // Directamente desde la clase
```

También pueden invocarse **como atributos** de un objeto, pero no tienen ningún acceso a los atributos propios de ese objeto.

```
Oj. sum_cls_attr(2)
```

Un **método de clase** puede llamar a otros métodos de su clase usando su primer parámetro.

```
@classmethod
```

```
def class_method_1(cls, param1, param2):
```

```
    ...
```

```
@classmethod
```

```
def class_method_2(cls, param1, param2):
```

```
    ...
```

```
    cls.class_method_1(param1, param2)
```

```
    ...
```

Un método de instancia puede llamar a un método de clase de su propia clase con el atributo `__class__` del objeto al que está vinculado, pero no al revés.

```
def instance_method(self):
```

```
    self.__class__.class_method_1(1, 2)
```

• USO DE MÉTODOS DE CLASE

Principalmente se usa para tratar los atributos de clase.

Otro uso importante es el de 'métodos fábrica' (factory methods), para crear objetos de la clase usando diferentes conjuntos de parámetros para inicializarlos.

MÉTODOS
DE
CLASE

Por ejemplo, si tenemos un programa que trabaje con ángulos en grados podemos usar un método de clase para que haga la conversión a grados si pasara un valor en radianes.

TEMA 5. Herencia

En POO, la herencia es un mecanismo que permite definir nuevas clases a partir de clases existentes.

HERENCIA

Las clases definidas de esta forma son subclases o clases derivadas. A la clase de la que derivan se le denomina superclase.

Las subclases heredan características y comportamientos de sus superclases. En general, cambian (overide) algunos aspectos y comportamientos e introducen nuevos. Permite crear jerarquías de clase.

En Python todas las clases heredan de forma implícita la clase `object`.

CLASE
OBJECT

INICIALIZACIÓN
DE
CLASES
DERIVADAS

Para hacer un método de inicialización en una subclase, se debe llamar al inicializador de la otra clase (el `__init__` de la superclase). Para ello, se usa el prefijo `'super()'`

E.g.

```
class Two(One):
```

```
    def __init__(self, attr1, attr2): # Se define el init de la clase
        super().__init__(attr1) # se llama al __init__ de la superclase y se
                                # le pasa el parámetro necesario.
```

ATRIBUTOS
DE
CLASE

Un objeto de una clase derivada es también un objeto de la clase base de la que esa clase deriva. El objeto tiene, tanto los atributos de instancia definidos en la subclase, como los definidos en la superclase, pero de la misma solo puede acceder a atributos públicos.

Existen 2 formas de saber si un objeto es instancia de una clase determinada:

→ `isinstance (object, class)`: Devuelve True o False

AVERIGUAR LA CLASE

→ `object.__class__`: `C1.__class__` → Devuelve la clase.

También podemos saber si una clase es subclase de otra con:

→ `issubclass (class1, class2)`: Devuelve True o False

CLASES

ABSTRACTAS

Una clase abstracta es una clase que contiene métodos abstractos, que son declarados, pero no implementados.

Estas clases deben derivar de la clase **ABC** (Abstract Base Class), y los métodos abstractos deben estar marcados con el decorador **@abstractmethod**.

```
from abc import ABC, abstractmethod
```

```
class Shape(ABC):
```

```
    @property
```

```
    @abstractmethod
```

```
    def area (self):
```

```
        pass
```

NOTA: En una clase abstracta no pueden crearse objetos. Solo sirven para definir un conjunto de métodos a modo de contrato, al que otras clases pueden adherirse siendo herederos, con lo que se obliga a implementar todos los métodos abstractos.

NOTA: Una subclase de una clase abstracta que no implemente todos los métodos abstractos, sigue siendo abstracta y no se puede instanciar.

NOTA: Una clase abstracta puede contener métodos no abstractos, así como otros atributos de datos.

El polimorfismo puede definirse como la capacidad de usar una misma interfaz y obtener comportamientos diferentes, dependiendo del tipo de los objetos en los que se use.

E.g.

```
x = 5 + 10 #15
```

```
greetings = "Hello" + "world" # 'Hello world'
```

En POO, el polimorfismo permite que una clase derivada pueda sustituir o redefinir (to override) un método de su superclase.

El polimorfismo se implementa mediante los mecanismos de sustitución de métodos (methods overriding) y sobrecarga (overloading).

POLIMORFISMO

OVERRIDING
O
SUSTITUCIÓN
DE
MÉTODOS

La sustitución de métodos es un mecanismo que permite a una subclase proporcionar su propia implementación de un método ya implementado en alguna clase heredada.

En Python, para sustituir un método, una subclase solo necesita tener un método con el mismo nombre, el cual oculta al correspondiente de la subclase.

C.g.

```
class BaseClass:
```

```
    def message(self):  
        return "String prueba"
```

```
class ClassOne (BaseClass):
```

```
    def message(self):  
        return "String 1x ClassOne"
```

Nota: se puede reutilizar el método que será sustituido siendo llamado a través del método `super()`. (E.g. `super().message()`)

POLIMORFISMO
Y
MÉTODOS
MÁGICOS

El polimorfismo se usa con frecuencia a los métodos mágicos. Los métodos mágicos es un método especial que una clase puede sustituir para dar un comportamiento adecuado a sus objetos.

Las clases nuevas casi siempre modifican el método `__init__` y `__str__`. Esto es un overriding, pero se puede acceder a los métodos mágicos de la clase padre, usando el `super()`.

DEFINIR
NUEVAS
EXCEPCIONES

Python dispone de muchas excepciones predefinidas, sin embargo, hay situaciones en las que necesitamos definir un nuevo tipo de excepciones, para un caso + específico.

Para definir un nuevo tipo de excepción, creamos una clase que herede, directa o indirectamente, la clase predefinida `Exception`.

AJUSTE DE
EXCEPCIONES
NUEVAS

Tmb pueden incluir atributos de datos y métodos, aunque se recomienda dejarlo lo + simple posible.

JERARQUÍA

Cuando se crea un módulo que pueda lanzar distintos tipos de excepciones, es común crear una clase base para las excepciones definidas en ese módulo, creando subclases de la misma para las distintas situaciones de error, formando una jerarquía.

- `DATEERROR`

 - `MonthError`

 - `DayError`

Tipo de polimorfismo que habilita que el mismo operador funcione de manera diferente dependiendo del tipo de datos a los que se aplique.

TIPOS DE OPERADORES

- Operadores de comparación
- Funciones y operadores unarios
- Operadores aritméticos binarios
- Operadores aritméticos reflejados.

Operadores de comparación:

Los siguientes operadores pueden redefinirse para establecer el funcionamiento de los operadores:

- ↳ `__eq__` → define el operador `==`
- ↳ `__ne__` → `!=`
- ↳ `__lt__` → `<`
- ↳ `__gt__` → `>`
- ↳ `__le__` → `<=`
- ↳ `__ge__` → `>=`

Ordenación total

No es necesario implementar todos los métodos mágicos de comparación, pues si añadimos a la clase el decorador `@functools.total_ordering`, basta con implementar el `__eq__` y uno de los siguientes: `__lt__`, `__le__`, `__gt__`, `__ge__`.

E.G.

```
from functools import total_ordering
```

```
@total_ordering
class Rational:
```

Nota: se carga el decorador `@total_ordering` de `functools` y se pone antes de la clase

Funciones y operadores unarios

- | | |
|--|---|
| ↳ <code>__pos__</code> → <code>+</code> (unario) | ↳ <code>__round__</code> → <code>round()</code> redondear |
| ↳ <code>__neg__</code> → <code>-</code> (unario) | ↳ <code>__floor__</code> → <code>floor()</code> [] |
| ↳ <code>__abs__</code> → función <code>abs</code> [absolute value] | ↳ <code>__ceil__</code> → <code>ceil()</code> [] |
| ↳ <code>__invert__</code> → <code>~</code> complemento bit a bit | ↳ <code>__trunc__</code> → <code>trunc()</code> trunca |

Operadores aritméticos binarios

tienen 2 operandos: Operando 1 operador Operando 2

- | | | |
|---|---|---|
| ↳ <code>__add__</code> → <code>+</code> | ↳ <code>__mod__</code> → <code>%</code> | ↳ <code>__and__</code> → <code>&</code> |
| ↳ <code>__sub__</code> → <code>-</code> | ↳ <code>__divmod__</code> → <code>divmod()</code> | ↳ <code>__or__</code> → <code> </code> |
| ↳ <code>__mul__</code> → <code>*</code> | ↳ <code>__pow__</code> → <code>**</code> | ↳ <code>__xor__</code> → <code>^</code> |
| ↳ <code>__floordiv__</code> → <code>//</code> | ↳ <code>__lshift__</code> → <code><<</code> | |
| ↳ <code>__div__</code> → <code>/</code> | ↳ <code>__rshift__</code> → <code>>></code> | |
- Nota: todos tienen `__i__` (self, other)

SOBRECARGA DE OPERADORES

• Operadores binarios reflejados.

Están pensados para que cuando se use un operador con sus operandos intercambiados (other <op> self en vez de self <op> other). Se aplican cuando operandos de distinto tipo.

Los nombres son los mismos que los operadores normales pero con una r al principio `--add--` $\rightarrow$ `--radd--`

• Sobrecarga de funciones

tipo de polimorfismo que permite usar un mismo nombre de función (o método) con diferentes parámetros en el mismo espacio de nombres. Python no permite esto, pero permite el uso de parámetros opcionales con valor definido por omisión.

E.g.

```
def hello (name = None):  
    if name != None:  
        print ('Hello' + name + "!!")  
    else:  
        print ('Hello')
```

~~Nota~~ Por tanto, en Python hay una especie de funciones polimórficas en vez de funciones sobre-cargadas.

Se puede invocar de dos formas, pasándole un nombre o no

• Polimorfismo basado en parámetros por omisión

Es lo explicado antes

• Polimorfismo basado en el tipo de parámetros.

Es el mismo concepto anterior pero en vez de comprobar si el parámetro se pasa o no se comprueba el tipo de este.

En Python, un iterador es un objeto que permite recorrer en secuencia un conjunto de elementos, obteniendo uno en cada iteración. Para esto se debe definir 2 métodos mágicos: `--iter--` y `--next--`.

```
class MyRange():
```

```
    def __init__(self, start=0, stop=0, step=1)
```

```
        self.... =
```

```
        .... =
```

```
    def __iter__(self):
```

```
        self.current = self.start
```

```
        return self.
```

```
    def __next__(self):
```

```
        if self.current <= self.stop:  
            return result = self.start
```

```
            self.current += self.step
```

```
            return result
```

```
        else:  
            raise StopIteration
```

ITERADORES

El método `__init__` establece los parámetros de iteración, el método `__iter__` prepara el objeto para empezar la iteración y lo devuelve. El método `__next__` devuelve el elemento actual de la secuencia y avanza el siguiente. La excepción `StopIteration` señala el final del recorrido.

• Funciones predefinidas `iter` y `next`.

Funciones que permiten recorrer cualquier objeto iterable.

↳ `iter(obj)` → pide a `obj` que devuelva un iterador `obj.__iter__()`

↳ `next(obj)` → pide el siguiente valor del iterador `it.__next__()`

Nota: Así funciona el for

• Generadores

En Python los generadores son mecanismos simples para definir iteradores. Tienen la apariencia de una función normal, con la diferencia de que la sentencia `return` es sustituida por la sentencia `yield`.

```
def range_generator(start, stop, step=1):
```

```
    current = start
```

```
    while current <= stop:
```

```
        yield current
```

```
        current += step
```

TEMA 6. PRUEBAS DE PROGRAMAS

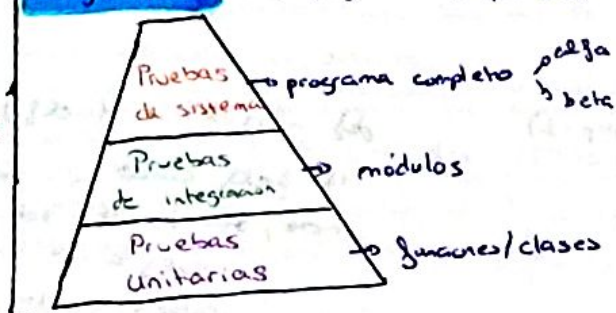
Para las pruebas de un programa se seleccionan unos casos de prueba que tienen mayor probabilidad para detectar errores. Cada caso tiene un conjunto de datos de entrada y un conjunto de resultados esperados, si el resultado final no se corresponde con este último → hay un defecto

Datos entrada → PROGRAMA → Datos salida == datos esperados

Antiguamente → 1° programar → 2° pruebas

Ahualmente → 1° pruebas → 2° programar (TDD)

DISCEN
GENERAL
PRUEBAS



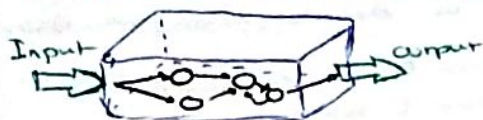
Nota: ante cualquier modificación del código se hacen pruebas de regresión (todas o parte de las ya ejecutadas)

TIPOS DE PRUEBAS

- PRUEBAS DE CASA BLANCA** Se basan en el conocimiento del código y su objetivo es probar todos los posibles caminos de ejecución. E.j. `if/else` → prueba donde se vaya por `if` y otra donde se vaya por `else`.
- PRUEBAS DE CASA NEGRA** Solo se conoce la interfaz del código, se limitan a comprobar que para todas las cosas de prueba las entradas suministradas generan los resultados esperados.

NOTA: Tmb se habla de pruebas de caja gris, que mezcla las 2 anteriores.

PRUEBAS DE CASA BLANCA

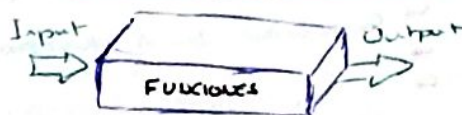


Para encontrar los diversos caminos por donde pasa el flujo del programa se usa la

Prueba del Camino Básico

1. Grafo del flujo de ejecución
2. Se calcula la complejidad ciclométrica
3. Se obtienen los caminos básicos
4. Se diseñan casos de prueba que recorra cada camino

PRUEBAS DE CASA NEGRA



Buscar entradas cuya probabilidad de causar fallos sea la + alta posible. Se seleccionan mediante 2 técnicas.

- Clases de equivalencia
- Prueba de valores límites

GRAFO DEL FLUJO DE EJECUCIÓN, COMPLEJIDAD CICLOMÉTRICA Y CAMINOS BÁSICOS

GRAFO DEL FLUJO DE EJECUCIÓN

Grafo orientado en el que el vértice representa o instrucciones o conjuntos de estas, o condiciones. Estos últimos son vértices predicados.

Complejidad ciclométrica y caminos básicos

Para las pruebas de caja blanca necesitamos caminos básicos, que son el conjunto mínimo de caminos que garantiza que cada arco del grafo aparece al menos en un camino. La manera más sencilla es empezar por el más corto.

El número de caminos necesarios se puede calcular en la complejidad ciclométrica así:

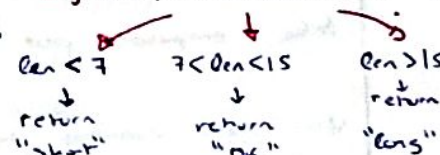
$$N^{\circ} \text{ caminos básicos} = N^{\circ} \text{ vértices predicados} + 1$$

Clases de equivalencia

Consiste en dividir las posibles entradas en particiones, es decir, en entradas para las que la función tiene un comportamiento equivalente.

NOTA: En el ejemplo habría que generar una prueba para cada grupo de datos (≤ 7), ($7 < n \leq 15$), (> 15)

E.j. Comprobar_Password (String):



CLASES DE EQUIVALENCIA Y PRUEBA DE VALORES LÍMITES

También existen las clases de equivalencia no válidas, E.g. en una clase que represente racionales, el denominador nunca podrá ser 0 → no es válido.

Prueba de los valores límites

Esta técnica complementa a las clases de equivalencia, pues lo que propone es que cuando estas sean rangos se deben comprobar los extremos de la clase. Específicamente el min, el sig al min, el máx, el anterior al máx y un valor central.

Las pruebas donde intervienen contenedores se complican un poco más (Ej.: diccionarios, conjuntos, listas, tuplas, string, etc.). Hay varias dimensiones a tener en cuenta.

PRUEBAS

DE

CONTENEDORES

1. TAMAÑO

- ↳ Contenedor vacío (tamaño 0)
- ↳ Contenedor tamaño n
- ↳ Contenedor con varios elementos (tamaño n)

2. ESPECIFICACIONES DEL PROGRAMA

Por ejemplo, si fuese un programa que busca un elemento habría que observar:

- ↳ Contenedor de n elementos valor buscado aparece 0 veces
- ↳ " " " " " " " " aparece 1 sola vez
- ↳ " " " " " " " " aparece $n-1$ veces
- ↳ " " " " " " " " aparece n veces

Además habría que tener en cuenta que si aparece n veces entonces habría que mirar la posición.

PRUEBAS

DE

CLASES

Las pruebas de clases son básicamente pruebas de sus métodos, que son funciones, al fin y al cabo. Luego, la estrategia a seguir es identificar sus propiedades y aquellas métodos como el `__init__` que, a priori, no dependen de otros. Desarrollamos pruebas por estos métodos. Una vez testeados los métodos básicos pasamos a las pruebas de los que dependen de ellos.

Los correedores de pruebas (test runners) permiten ensayar conjuntos de casos de manera independiente, produciendo un informe final de los casos fallidos y exitosos.

CORREDORES

DE

PRUEBAS

Esta opción es mucho más rentable que definir una función por prueba "manualmente" pues al ejecutar varias funciones el programa se pararía en la primera excepción.

Unittest es el framework oficial de pruebas de Python.

• TestCase

UNITTEST

Unittest ofrece una clase llamada `TestCase`. Para usarla se crea una clase heredera o implementa cada caso de prueba como un método cuyo nombre debe empezar por `"test"`. Siempre 1 método $\rightarrow$ 1 prueba.

Una vez escritas todas las pruebas se ha de llamar al método `main()` de `unittest`, que ejecuta todos los test en orden alfabético.

MÉTODOS

DE

ASSERT

Para hacer las comprobaciones se pueden usar los asserts de `TestCase`. `assertEqual()`, con 3 parámetros: los valores a comparar y un tercero, opcional, con un mensaje explicativo.

Ej.

`def test_no_evens(self):`

`"""una lista sin n° pares"""`

`result = sum_evens([1, 3, 5, 7])`

`self.assertEqual(result, 0, "sin pares")`

Hay métodos de aserto específicos, además de `assertEqual()`.

MÉTODOS

`assertEqual(a, b)`

`assertNotEqual(a, b)`

`assertTrue(x)`

`assertFalse(x)`

`assertIs(a, b)`

`assertIsNot(a, b)`

`assertIsNone(x)`

`assertIsNotNone(x)`

`assertIn(a, b)`

`assertNotIn(a, b)`

`assertIsInstance(a, b)`

`assertNotIsInstance(a, b)`

COMPRUEBA

`a == b`

`a != b`

`bool(x) is True`

`bool(x) is False`

`a is b`

`a is not b`

`x is None`

`x is not None`

`a in b`

`a not in b`

`isinstance(a, b)`

`not isinstance(a, b)`

Nota: Todos estos métodos admiten un tercer parámetro extra opcional para cuando el aserto falle.

Además de los anteriores, existen otros asertos que comprueban que el código probado produzca una excepción, un warning o un mensaje de log. Se suelen usar en el contexto de una cláusula `with`.

E.g.

```
def test_empty_list(self):
```

```
    """ Una lista vacía """
```

```
    with self.assertRaises(ValueError, msg="No se lanza la excepción esperada"):
```

```
        result = sum_evens([3])
```

`assertRaises()` comprueba que la excepción esperada se produce. Además de este, otros métodos similares son: `assertRaisesRegex()`, `assertWarns()`, `assertWarnsRegex()` y `assertLogs()`.

El propio framework ofrece algunos métodos de aserto + específicos.

MÉTODOS

`assertAlmostEqual(a, b)`

`assertNotAlmostEqual(a, b)`

`assertGreater(a, b)`

`assertGreaterEqual(a, b)`

`assertLess(a, b)`

`assertLessEqual(a, b)`

`assertRegex(s, r)`

`assertNotRegex(s, r)`

`assertCountEqual(a, b)`

COMPRUEBA

`round(a-b, 7) == 0`

`round(a-b, 7) != 0`

`a > b`

`a >= b`

`a < b`

`a <= b`

`r.search(s)`

`not r.search(s)`

`a + b` no misma denotación en n^0 independiente del orden.

TEST
FIXTURE

La clase TestCase proporciona 2 métodos: .setUp() y .tearDown() que se ejecutan respectivamente antes y después de cada prueba. Usado para acceder a ficheros, bases de datos, etc. y luego cerrar las conexiones.

También esta .setUpClass() y .tearDownClass() que son lo mismo que las anteriores pero se abre una única vez y se cierra una única vez, antes y después de ejecutar todas las pruebas.

TEST
SUITE
EXECUTION
OF
TEST

Ya se vio que el unittest.main() ejecuta los test del módulo en que se encuentre, si es el principal.

Realmente el main() crea una test suite con el conjunto de test del módulo y un test runner para ejecutarlos.

Una test suite es básicamente un conjunto de test que se quieren ejecutar juntos. Los test que la componen pueden pertenecer a distintas clases heredadas de TestClass y a distintos métodos.

Eg.

```
import unittest
import sampletest
```

```
def suite():
```

```
    Suite = unittest.TestSuite() # unittest.TestSuite() crea una suite
    Suite.addTest(sampletest.SumEvenTest('test-no-evens')) # addTest añade una prueba
    Suite.addTest(sampletest.SumEvenTest('test-all-evens'))
    return Suite
```

```
if __name__ == '__main__':
```

```
    runner = unittest.TextTestRunner() # crea el test runner con TextTestRunner()
    runner.run(suite()) # lo corre con .run(suite())
```

Los métodos main y run muestran los resultados del test en consola. El método run puede asignarse a una variable para manejarlo programáticamente (guardar un TestResult). Para hacer esto con main hay que poner el parámetro exit=False.

CLASE
TESTLOADER

Esta clase permite crear una TestSuite.

loader = unittest.TestLoader()

→ Crea una suite a partir del nombre de un módulo:

```
suite1 = loader.loadTestsFromName("sampletests")
```

→ Crear suite a partir del nombre de una TestClass:

```
suite2 = loader.loadTestsFromName("sampletests.SumEvenTest")
```

→ Crear suite a partir del nombre de una test-case:

```
suite3 = loader.loadTestsFromName("sampletests.SumEvenTest.test-no-evens")
# Para añadir varios crea una lista y un repetido el patrón y cambiando el nombre.
```

Además:

• loadTestFromTestCase() • loadTestsFromModule() • discover("...")

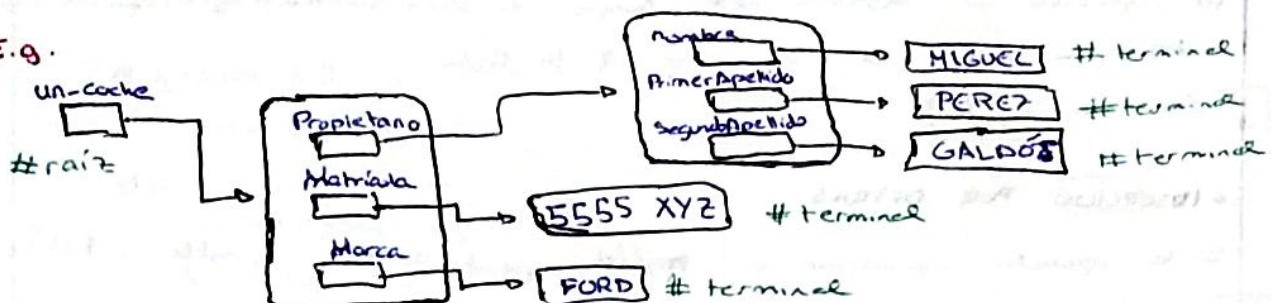
↳ Suites dentro de suites.
• suite.addTest(suite1)

TEMA 7. ESTRUCTURAS ENCADENADAS

En Python cuando se inicializa una variable asignándole un objeto de una clase determinada lo que la variable almacena es la dirección.

Los diferentes atributos de datos de un objeto son a su vez referencias a los objetos que representan los valores de esos atributos.

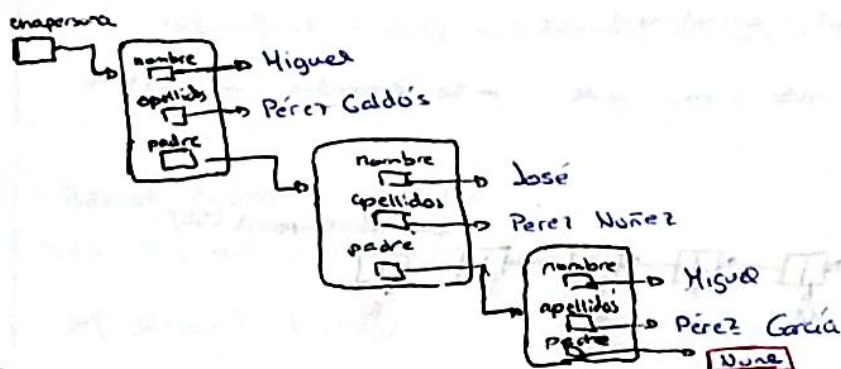
E.g.



De esta manera los objetos se encadenan unos con otros formando estructuras complejas.

Una estructura encadenada está formada por un conjunto de objetos, generalmente llamados nodos, que se referencian unos a otros. Debe existir al menos un objeto inicial (en el ejemplo un-coche) y un objeto terminal que no referencia a otro.

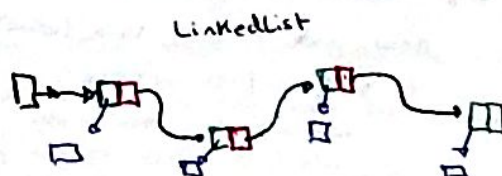
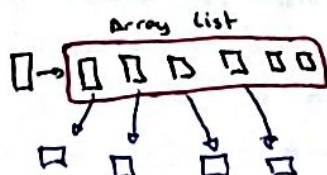
Si un objeto de clase X, tiene campos que son referencias a otros objetos, podrían ser referencias a otros objetos de la misma clase X. Por ejemplo, si se añade un atributo 'padre' a la clase 'Persona', siendo 'padre' una referencia a otro objeto de 'Persona', se crea una estructura recursiva.



Nota: en la praxis esta estructura no puede ser infinita.

Las estructuras encadenadas permiten crear estructuras complejas y flexibles. Por ello, se suele implementar en contenedores.

Una comparación podría ser las listas de Python (que son arrays dinámicos, es decir, guardan porción antigua de memoria) con un contenedor implementado mediante estructuras encadenadas, llamémosle LinkedList.



Nota: en el array si la memoria está muy fragmentada, no se podrá crear o habrá que reorganizarla.

Esto no ocurre con los LinkedList, pues no se necesita almacenamiento continguo. Aunque también tiene una desventaja: En una lista se puede acceder fácilmente a un elemento con un índice i ya que al ser continguo solo hay que sumar la dirección del comienzo del array el valor de multiplicar el tamaño de cada referencia por el índice), mientras que en una linked list tienes que pasar por todos los predecesores.

Además los nodos de una linked list requieren espacio extra para almacenar la referencia al siguiente nodo. Aunque es cierto que los arrays reservan espacio extra para facilitar el crecimiento de la lista.

• INSERCIÓN POR DETRÁS

Es la operación equivalente a: `mylist.append(value)` [`mylist` → `list`].

En una lista encadenada esto puede ser costoso si hay que recorrer toda la lista.

E.g.

```
def append(self, value):
```

```
    new_node = self.Node(value) # se crea nodo con next_node = None
```

```
    if self.first == None:
```

```
        self.first = new_node.
```

```
    else: # Busca el último, será el que tenga como next_node = None
```

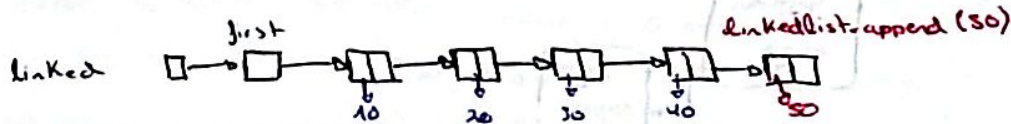
```
        current = self.first
```

```
        while current.next_node != None:
```

```
            current = current.next_node
```

```
        current.next_node = new_node # se engancha al último
```

```
        self.len += 1
```



• INSERCIÓN POR DETRÁS (2)

Otra forma de hacerlo es añadiendo un atributo `self.last` que guarda la referencia al último, lo cual reduce el coste de recorrer toda la lista aunque añade el coste de tener que actualizar el atributo.

E.g.

```
def append(self, value):
```

```
    new_node = self.Node(value)
```

```
    if self.first == None:
```

```
        self.first = new_node
```

```
    else:
```

```
        self.last.next_node = new_node
```

```
    *
```

```
* self.last = new_node
```

```
self.len += 1
```

• INSERCIÓN EN MEDIO

Sería como hacer un `insert()` en una lista de Python:

```
def insert(self, where, value):
```

```
    if where == 0:
```

```
        self.insert_at_first(value) # vemos el método + adelante
```

```
    elif where == len(self):
```

```
        self.append(value)
```

```
    elif where < 0 or where > len(self):
```

```
        raise IndexError
```

```
    else:
```

```
        current = self._first
```

```
        current_pos = 1
```

```
        while current_pos < where:
```

```
            current = current.next_node
```

```
            current_pos += 1
```

```
        current.next_node = self._Node(value, current.next_node)
```

```
        self._len += 1
```



• INSERCIÓN POR DELANTE

Insertamos al principio de la lista.

```
def insert_at_first(self, value):
```

```
    self._first = self._Node(value, self._first)
```

```
    self._len += 1
```

• BORRAR ELEMENTO INICIAL

Solo hay que cambiar la referencia del primer nodo.

```
def delete_first(self):
```

```
    if self._first != None:
```

```
        self._first = self._first.next_node
```

```
        self._len -= 1
```

```
    if self._len == 0:
```

```
        self._last = None
```



Nota: en arrays esta operación es costosa porque hay que desplazar el resto de elementos

• BORRAR AL FINAL

Para borrar por el final, a menos que solo haya un elemento, se ha de encontrar el penúltimo y convertirlo en el último. En este caso la referencia al `self._last` no disminuye el coste, pues hay que recorrer la lista.

def delete_last(self):

if self._len == 0:

if self._len == 1:

self._last = self._first = None

si solo tiene 1 elemento

se queda vacía.

else:

prev = self._first

current = self._first.next_node

while current != self._last:

Buscamos el penultimo.

prev = current

current = current.next_node

prev.next_node = None

Convertimos el penultimo en el ultimo

self._last = prev

self._len -= 1

• BORRAR POR POSICIÓN

def delete(self, index):

if index < 0:

index = self._len + index

if index in range(self._len):

if index == 0:

self._first = self._first.next_node

self._len -= 1

if self._len == 0:

self._last = None

else:

prev = self._first

current = self._first.next_node

current_index = 1

while current_index != index:

Buscamos el node anterior al que queremos borrar.

prev = current

current = current.next_node

current_index += 1

prev.next_node = current.next_node

saltamos el node a borrar

if current == self._last:

por si es el ultimo node

self._last = prev

self._len -= 1

else:

raise IndexError

• ITERACIÓN

Cabe recordar que la clase LinkedList es un contenedor de elementos implementado mediante una lista enlazada.

```
class LinkedList:
```

```
    class Node:
```

```
        def __init__(self, value, next_node = None):
```

```
            self.value = value
```

```
            self.next_node = next_node
```

```
    def __init__(self):
```

```
        self._first = None
```

```
        self._len = 0
```

```
    def _len_(self):
```

```
        return self._len
```

Los contenedores predefinidos en Python (listas, diccionarios, ...) son iterables.

Para ello, podemos implementar el método mágico `__iter__()`, con una función generadora.

```
    def __iter__(self):
```

```
        current = self._first
```

```
        while current != None:
```

```
            value = current.value
```

```
            current = current.next_node
```

```
            yield value
```

LISTAS
ENLAZADAS

FP2: JAVA

En Java un programa consta como mínimo de una clase, que debe tener un método de clase llamado main.

Ej.

```
class HolaMundo {  
    public static void main(String[] arg) {  
        System.out.print("¡Hola, mundo!");  
    }  
}
```

La clase puede llamarse como se quiera

El método principal no solo debe llamarse main:

- Las variables y métodos declarados en una clase pueden tener 3 posibles **modificadores de visibilidad**: **public**, **protected**, **private**.
- El modificador **static**, que indica que es un **método de clase**, no de instancia.
- Declarar el **tipo de resultado** que devuelve el método con un return. El void significa que no devuelve nada (lo que a Python sería un None).
- El método principal debe poseer el argumento **arg**, de tipo **String[]**, que designa un array (similar a una lista) de strings. Sirve para darle datos al programa.

NOTA: Todas las características anteriores son necesarias, pero no exclusivas, del método principal main, que es el punto de entrada del programa. Mas no todas las clases lo necesitan. Lo normal es tener varias clases y una sola clase especial que sí lo incluye para inicializar el programa.

• LLAVES, SANGRADO Y PUNTO Y COMA.

En Java, una secuencia de instrucciones que pertenece a una **clase**, **método** o **estructura de control**, se encierra entre llaves **{}**.

Tamb se sangra, pero esto se hace por convención para mejorar la legibilidad del programa.

Las instrucciones 'simples' deben acabar siempre en punto y coma (;) para indicarle al compilador que ahí termina la instrucción.

• SALIDA POR CONSOLA

En Java, para mostrar algo por consola se usa el método `System.out.println()`. Este inserta un salto de línea detrás del mensaje, esto se puede evitar usando `System.out.print()`.

NOTA: Ambos métodos no admiten + de un parámetro, por lo que no hay separadores.

• VARIABLES

En Java, una variable se crea declarando su **tipo** y su **nombre**, pudiéndole asignar un valor, o no, en el momento de la declaración. Deberá asignársele un valor antes de su primer uso. Por convención, los nombres de las variables empiezan por una letra minúscula, usar letras minúsculas para formar palabras, y, en caso de nombres compuestos se pone la primera letra de cada palabra (excepto la primera) en mayúscula.

Ej: mixedCase.

```
int intVar = 100;
```

```
float floatVar = 100.80f; # si no se pone f → se considera de tipo doble
```

```
String strVar = "Hola";
```

NOTA: El tipo declarado para una variable no se puede cambiar (no puede recibir un valor de tipo diferente). El tipo de una var solo se pone la primera vez que aparece.

Se puede declarar una variable sin asignarle un valor explícitamente; quedando con un valor imprevisible o con uno establecido por omisión.

Si se le pone a la variable el modificador final (antecediendo al tipo) impide modificarla posteriormente.

• TIPOS DE DATOS PRIMITIVOS

↳ 5 tipos de n° enteros, de menor a mayor rango: byte, char, short, int y long.

↳ 2 tipos n° reales: float y double

↳ Otros tipos: boolean (T/F) y void (vacío)

NOTA: También existe null, que es como el None de Python (ref a objetos que no apuntan a nada).

Los tipos byte y char se usan también para representar caracteres (texto).
Char (x = 'a');

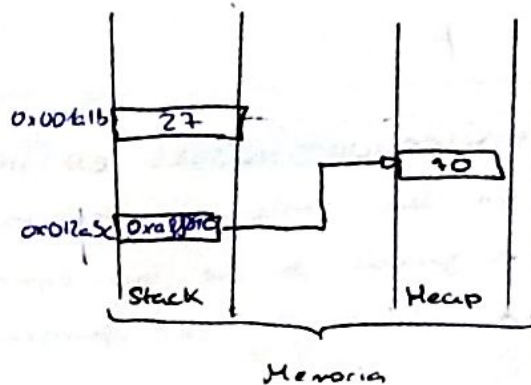
El tipo String, representa secuencias de caracteres, no se considera primitivo, pero tiene un tratamiento particular.

• VALORES Y REFERENCIAS

En Java, los tipos de datos **no primitivos** se implementan como referencias a objetos (de alguna clase) en el heap, los tipos de datos primitivos almacenan directamente su valor. No obstante, a cada tipo primitivo se asocia una clase envoltorio (wrapper) equivalente para cuando hace falta representar ese tipo de valores en formato objeto: Byte, Character, Short, Integer, Long, Float, Double, Boolean.

```
int a = 27;
Integer b = 70;
```

Mem	DIR
a	0x001a1b
b	0x012a2c



• ENTRADA BÁSICA DE DATOS

En Java, hay varias formas de entrar datos (input). Una de las más sencillas es usando un objeto de la clase Scanner.

Ej.

```
import java.util.Scanner; #1

class Input {
    public static void main (String[] args) {
        Scanner input = new Scanner (System.in); #2
        System.out.print("Dame un número entero: ");
        int number = input.nextInt() #3
        System.out.println("Teclasteó " + number);
    }
}
```

PASOS

1. Importar java.util.Scanner
2. Crear objeto de tipo Scanner con System.in.
3. Usar el método adecuado del objeto, dependiendo del tipo de datos que se quiere leer (.nextInt(), .nextFloat(), .nextBoolean(), ...) Strings con .nextLine().

• SELECCIÓN BINARIA

En Java, la sentencia if-else permite ejecutar una secuencia u otra de instrucciones, en función de una condición.

Ej.

```
if (a > b) {
    System.out.print("a es mayor");
} else {
    System.out.print("b es mayor o igual");
}
```

NOTA: La cláusula else puede omitirse.

NOTA: La condición del if va entre paréntesis

CONTROL
DE
FLUJO

• ENCADENAMIENTOS ELSE-IF

En Java no existe la sentencia `elif`, así que se hace encadenando `if... else if... else`.

Ej:

```
if (~) {  
    ...  
} else if (~) {  
    ...  
} else {  
    ...  
}
```

• SELECCIÓN MÚLTIPLE EN JAVA (SWITCH - CASE)

En Java existe una sentencia especial para seleccionar un curso de acción en función de que una expresión discriminante tome un valor concreto dentro de un conjunto de opciones.

Ej:

```
switch (day) {  
    case 1: System.out.println("lunes");  
        break; // o Sale, si no se pone sigue el siguiente caso  
    case 2: " " " ("martes");  
        break;  
    case 3: " " " ("miércoles");  
        break;  
    default: " " " ("no es un día"); // Casos no indicados  
        break;  
}
```

• SENTENCIA FOR

En Java, podemos usar una sentencia `for` para recorrer una secuencia de valores:

Ej:

```
String[] names = {"Juan", "María", "Pedro", "Antonio"}; // array de Strings  
for (String name : names) {  
    System.out.println(name);  
}
```

• SENTENCIA FOR CON RANGO

En Java, se puede usar una sentencia `for` para recorrer un rango de valores:

```
for (int i = 0; i < 10; i++) {  
    System.out.println(i);  
}
```

Ojo: El control de la sentencia `for` va entre paréntesis y se divide en tres partes por `;`.

- 1 Variable de control con valor inicial.
- 2 Condición de iteración; termina el `for` si no se cumple.
- 3 incremento/decremento de variable de control.

• SENTENCIA WHILE

En Java, se puede usar la sentencia while para ejecutar un bucle controlado por una condición booleana entre paréntesis

```
int i = 0
while (i < 5) {
    System.out.println(i);
    i++;
}
```

CONTROL
DE
FLUJO

• FUNCIONES

En Java, no existen funciones libres, todas son métodos de instancia o de clase. Hay que declarar tanto el tipo de los parámetros como el del resultado que devuelve la función.

```
class Main {
    public static int fun (int param1, int param2) {
        return param1 + param2;
    }
    public static void main (String[] arg) {
        System.out.println (fun(10, 5));
    }
}
```

El modificador
'static' indica
que es un método
de clase.

En Java, el tipo de datos básicos para representar secuencias de valores son los arrays. Todos los elementos de un array han de ser del mismo tipo, que debe declararse. Para ello hay que poner el tipo de los elementos seguido por corchetes.

```
int[] arrayOfInt;
```

ARRAYS

Se puede inicializar un array asignándole una secuencia de valores, separados por comas y encerrados entre llaves:

```
int[] arrayOfInt = { 10, 15, 20, 30, 40 };
```

Tmb se puede inicializar creando un objeto array de un tamaño determinado con el operador new:

```
int[] arrayOfInt = new int[5] # Los elementos se inicializan todos con 0, false o  
null, de acuerdo con su tipo. (ste caso 0)
```

Los arrays no pueden cambiar de tamaño (ha vez creados (como listas de Python) pero los elementos pueden modificarse (como las listas de Python).

Igual que en Python, los elementos de un array en Java se numeran con índices comenzando en el 0. Para acceder a su valor es como en Python:

```
int value = arrayOfInt[3]; // Value = 9, arrayOfInt[3] = 10
```

Para cambiar un valor, se pone esa expresión a la izquierda de una asignación, igual que en las listas de Python.

```
arrayOfInt[3] = 10;
```

El número de elementos de un array se puede conocer accediendo al atributo **length**

```
int size = arrayOfInt.length;
```

• RECORRIDO DE UN ARRAY

Los algoritmos para recorrer un array son análogos a los usados en Python

Con índices

```
class Main {  
    public static void main (String[] args) {  
        int[] myArray = {10, 15, 20, 35, 40};  
        for (int i = 0; i < myArray.length; i++) {  
            System.out.println(i);  
        }  
    }  
}
```

sin índices

```
class Main {  
    public static void main (String[] args) {  
        int[] myArray = {10, 15, 20, 35, 40};  
        for (int value : myArray) {  
            System.out.println(value);  
        }  
    }  
}
```

• ESTRUCTURAS MULTIDIMENSIONALES

En Python tenemos listas de listas. En Java tenemos arrays de arrays

```
int[][] myArray = {  
    {10, 15, 20, 30, 40},  
    {12, 14, 16, 18},  
    {11, 17, 23, 24, 31},  
};
```

Nota: Hay que poner en la declaración tantos niveles de corchetes como dimensiones tengamos.

Para acceder a un elemento es igual que en Python: `myArray[i][j]`

Para el recorrido de estas estructuras se hace como en Python, usando tantos **bucles** como dimensiones haya. Independientemente de si se recorre por **índices** o **valores**.

• LA CLASE ARRAYS (1)

La clase `Array` (del paquete `java.util`) ofrece funciones de clase para operaciones útiles con arrays. `# java.util.Arrays;`

- El método `sort()` ordena de menor a mayor una array. Se le puede especificar un rango de índices para ordenar solo los elementos comprendidos en él.

```
import java.util.Arrays;
...
// array1 = { 80, 70, 60, 50, 40, 30, 20, 10 }
Arrays.sort(array1, 2, 5) // { 80, 70, 40, 50, 60, 30, 20, 10 }
Arrays.sort(array1) // { 10, 20, 30, 40, 50, 60, 70, 80 }
```

- El método `binarySearch()`, dado un array ordenado y un valor, busca el valor en el array haciendo uso de un algoritmo de búsqueda binaria y devuelve su posición. Si no lo encuentra devuelve el valor $-(\text{punto de inserción})-1$, siendo 'punto de inserción' la posición donde debería estar.

- El método `copyOf()` copia una array, creando una nueva con una longitud especificada. Si la longitud especificada es menor que la del array original, lo trunca, y si es mayor, rellena con el valor de inicialización del tipo de los elementos. `#(0|false|null)`

- El método `copyOfRange()` copia un trazo de una array especificado por un rango de índices, creando un nuevo array.

• LA CLASE ARRAYS (2)

- El método `equals()`, devuelve true si los 2 arrays que comparan son iguales.

Note: Cuando los elementos de los arrays son objetos (no valores primitivos), `equals()` solo hace una comparación superficial, para una comparación en profundidad se use el método `deepEquals()`.

- El método `fill()` rellena los elementos de un array con un valor determinado. Se puede especificar un rango de índices a rellenar.

- El método `toString()` devuelve un objeto de tipo `String` que representa el contenido del array que se le pasa.

En Java los objetos de clase String representan secuencias de caracteres, codificados en UTF-16. Se puede crear una encerrando caracteres entre **comillas dobles ("")**.

Ex. String string1 = "Hola mundo";

Las **comillas simples ('')** se reservan para los literales de tipo char (representan un carácter individual). Ex. char ch1 = 'a';

STRINGS

Una String vacía se puede crear:

↳ String empty1 = "" # Dobles comillas sin nada

↳ String empty2 = new String(); # Crea un nuevo objeto de clase String sin nada.

También puede crearse una string convirtiendo un valor de otro tipo:

Ex.

int number = 10;

String numberStr = ((Integer) number).toString();

Para aplicar el toString()
se ha convertido el valor
number del tipo int a la
clase Integer.

• SECUENCIAS DE ESCAPE

En Java, se usan secuencias que comienzan con una barra invertida (backslash) para incluir ciertos caracteres especiales en una string.

\n	Nueva línea: coloca el cursor de la pantalla al inicio de la siguiente línea.
\t	Tabulador horizontal: desplaza el cursor hasta la siguiente posición de tab
\r	Retorno de carro: coloca el cursor de la pantalla al inicio de la línea actual.
\"	Carácter de doble comilla.
\\	Carácter de backslash.

• LONGITUD DE UNA STRING

En Java, la longitud de una string se puede averiguar usando el método .length().

Ex.

String s1 = "esto es una string";

System.out.println(s1.length()); //18

• COMPARACIÓN DE STRINGS

En Java, NO se usan los **operadores relacionales** (<, <=, ==, !=, >=, >), porque comparan las referencias, no las strings. Para ver si 2 strings son iguales se usa el **método equals()**.

NOTA: Si no se quiere distinguir entre mayúsculas y minúsculas, se usa el método `equalsIgnoreCase()`.

Para Comparar si una string es menor, igual o mayor que otra, usamos el método `compareTo()`. Devuelve cero si las strings comparadas son iguales, un número positivo la primera es mayor que la segunda y negativo en otro caso.

Ex.

```
System.out.println(s1.compareTo(s2)); //6
System.out.println(s2.compareTo(s1)); // -6
String s1 = "esto es una string";
String s2 = "esto es una string";
```

Recordatorio: Las strings se comparan lexicográficamente, siendo más grande la que este + atrás en el diccionario

El número indica cuántas letras hay de diferencia ante la letra que difiere porque se comparan letra a letra

STRINGS

NOTA: Existe también `compareToIgnoreCase()` si se quiere ignorar la diferencia entre mayúsculas y minúsculas al comparar.

• CONCATENACIÓN DE STRINGS

Se pueden concatenar strings usando el operador `+` o el método `concat()`.

• ACCESO A CARACTERES Y SUBSTRINGS

En Java, se puede acceder a un carácter individual de una string usando el método `charAt()` con su índice de posición como parámetro; el resultado es un carácter (`char`), no una string.

Ex.

```
String s1 = "Hola mundo";
char c1 = s1.charAt(3); // 'a'
```

Además, se puede copiar una substring de una string usando el método `substring()`. Si se omite el segundo parámetro, copia hasta el final.

• CONTENCIÓN Y BÚSQUEDA (I y II)

- `contains()`: Se le pasa una substring por parámetro y si está a la string devuelve `true`.
- `indexOf()`: Localiza la primera aparición de una substring, devolviendo el índice. Si no la encuentra devuelve `-1`. Como segundo parámetro se le puede pasar un índice desde donde comenzar.
- `lastIndexOf()`: Igual que el anterior pero busca la última.

• OTRAS OPERACIONES

- `s.startsWith(prefix)`
- `s.endsWith(suffix)`
- `s.replace(old, new)`
- `s.replaceFirst(regex, new)`
- `trim()` → Elimina los espacios
- `split(regex)`
- `toLowerCase()`
- `toUpperCase()`

Java tiene un paquete llamado `java.util.regex` para trabajar con expresiones regulares. Este paquete ofrece 2 clases: `Matcher` (objetos para manipular expresiones regulares) y `Pattern` (objetos de expresión regular precompilada); y una excepción -o `PatternSyntaxException`, que indica una expresión regular mal formada.

Ex.

```
import java.util.regex.Matcher;
import java.util.regex.Pattern;

class Main {
    public static void main (String[] args) {
        String text = "una fecha 12-12-20 y otro 01-10-2019";
        Pattern pattern = Pattern.compile ("\\d{2}-\\d{2}-\\d{4}");
        Matcher matcher = pattern.matcher (text);

        while (matcher.find()) {
            System.out.println (matcher.group(0));
        }
    }
}
```

• BÚSQUEDA DE COINCIDENCIAS

En Java, para encontrar y procesar todas las coincidencias en un texto se usa el método `find()` de la clase `Matcher`, que devuelve en cada iteración la siguiente coincidencia encontrada. La clase `Matcher` ofrece varias herramientas. Cuando no hay + coincidencias, el resultado del método `find()` es falso.

NOTA: A diferencia del `finditer()` devuelve una secuencia con todas las coincidencias, mientras que en Java cada llamada a `find` devuelve una única coincidencia.

• SUSTITUCIÓN

El método `replaceAll()` de la clase `Matcher` sirve para sustituir las coincidencias del patrón en la string de búsqueda por un nuevo valor:

Ex.

```
import java.util.regex.Matcher;
import java.util.regex.Pattern;
```

```
class Main {
    public static void main (String[] args) {
        // ...
        String newText = matcher.replaceAll ("DATE");
    }
}
```

En Java iniciamos la definición de una nueva clase de objetos con la palabra `class` seguida del nombre de la clase. Ex. `class MyClass {}`

Para crear un nuevo objeto de la clase, usamos una expresión constructora formada por la palabra reservada `new`, seguida del nombre de la clase, seguida de los parámetros de inicialización.

Ex.

```
MyClass myObject = new MyClass ();
```

• ATRIBUTOS DE DATOS DE OBJETO, INICIALIZACIÓN

En Java los atributos de un objeto se declaran en el **cuerpo** de su clase y se inicializan, en su caso, con los valores pasados al constructor de la clase que se haya utilizado. Un constructor es un método especial de inicialización que tiene el **mismo nombre** que la clase, **no devuelve nada** y no tiene **modificadores** (salvo el de acceso).

Ex.

```
class MyClass {
```

```
    int attribute1; } atributos (de 'clase', no de instancia); Realmente no  
    int attribute2;
```

```
    public MyClass (int value1, int value2) { // Constructor.
```

```
        this.attribute1 = value1;
```

```
        this.attribute2 = value2; // self. en Java
```

NOTA: En Java se pueden tener varios constructores, con el requisito de que las listas de parámetros difieran en n° o tipo.

Además, en Java no se pueden añadir atributos dinámicamente como en Python.

• MÉTODOS DE OBJETOS

En Java un método de objeto, o método de instancia, es una función que se define en el cuerpo de una clase **sin usar modificador static**, el cual se usa para definir atributos de clase. A diferencia de Python en Java no existen funciones "independientes", que no sean métodos de instancia o clase.

Ex.

```
class MyClass {
```

```
    ...
```

```
    public int MyMethod (int param) {
```

```
        return (this.attribute1 + this.attribute2) / param;
```

```
    }
```

```
}
```

• VISIBILIDAD

En Java los atributos pueden llevar uno de entre 3 modificadores de visibilidad o no llevar ninguno.

↳ **public**: visible por todo el mundo.

↳ **protected**: visible para la clase, sus clases derivadas y clases del mismo package.

↳ **private**: visible solo para la clase.

Un atributo sin modificador de visibilidad es visible para la clase y las clases del mismo package pero no para las clases derivadas pertenecientes a otro package.

NOTA: Las clases no pueden llevar los modificadores **private** o **protected** y en un archivo pueden definirse varias clases pero solo puede haber una con el modificador **public** y debe tener el mismo nombre que el archivo.

ENCAPSULAMIENTO DE DATOS

Como ya se vio con Python, es recomendable proteger los datos de los objetos usando las técnicas de encapsulamiento para ocultar los atributos que los representan y proporcionar mecanismos que controlen como se pueda acceder y modificar.

Por ello, en Java, los atributos de datos de un objeto deberían llevar el modificador **private**. Una vez ocultos se escriben **getters** y **setters**. Un **getter** es un método sin parámetros cuyo nombre típicamente se forma con el prefijo **get** seguido del nombre del atributo con la primera letra en mayúscula. Un **setter** es un método con un parámetro y cuyo nombre se forma igual con el prefijo **set**.

Ex.

```
class Person {  
    private String name; // atributo  
  
    Person (String name) { // Constructor  
        this.name = name;  
    }  
  
    public String getName() { // Setter  
        return this.name;  
    }  
  
    public void setName (String newName) { // Setter  
        this.name = newName;  
    }  
}
```

Nota: Java no tiene **@property**, pero permite definir una propiedad de solo escritura.

ATRIBUTOS DE CLASE

Datos que no pertenecen a un objeto concreto, sino que son comunes a todos los objetos, o métodos que no se asocian a un objeto concreto y sirven para manejar los datos de clase directamente.

En Java, los atributos de clase, sean variables o métodos, se distinguen por llevar el modificador **static**.

En Java, para indicar que una clase hereda de otra, se pone la cláusula **extends** seguida del nombre de la superclase.

Ex.

```
class One {}  
class Two extends One {}
```

Nota: La subclase tiene acceso a todos los atributos disponibles en su superclase excepto los que tengan el modificador **private**.

INICIALIZACIÓN DE SUBCLASES

En Java, cuando inicializamos el constructor de una subclase, normalmente tenemos que llamar al constructor de su superclase usando el identificador **super**.

CLASES
y
OBJETOS

HERENCIA

Ex.

```
class Two extends One { // Subclase  
    int attr2;
```

```
    public Two (int value1, int value2) {  
        super (value1);  
        this.attr2 = value2;
```

HERENCIA

El constructor / inicializador de la subclase debe recibir, además de los propios, los parámetros a transferir (normalmente como primera instrucción) al constructor / inicializador de la superclase, a menos que los calcule a partir de los propios.

• SUSTITUCIÓN DE MÉTODOS (OVERRIDING)

En Java para sustituir un método en una subclase, el nuevo método debe tener **mismo nombre y parámetros** (misma ~~signature~~), preferentemente con la anotación **@Override**, resultando oculto el correspondiente de la superclase.

Ex

```
class BaseClass { // Superclase  
    public String message () {  
        return "This is a message from Base Class";
```

```
    }
```

```
class ClassOne extends BaseClass { // Subclase  
    @Override  
    public String message () {  
        return "This is a message from ClassOne";
```

Nota: Se puede acceder al método de BaseClass con un objeto de ClassOne usando super.
// super.message()

• JERARQUÍA DE CLASES

En Java todas las clases heredan de forma implícita de una clase base llamada **Object**. La clase Object define una serie de métodos con un comportamiento por omisión que se pueden sustituir (override).

En Java una clase solo puede tener **1 superclase**.

En Java, si una clase tiene el modificador **final**, no se pueden crear subclases que hereden de ella.

• MÉTODOS HEREDADOS DE OBJECT

En Java, heredar de la clase Object proporciona los siguientes métodos que pueden sustituirse (overriding).

→ clone () → Crea y devuelve copia del objeto this.

→ equals (Object obj) → indica si otro objeto es igual al objeto this.

→ finalize () → lo llama el garbage collector cuando el objeto this ya no es accesible al programa, antes de liberar su memoria.

→ getClass () → devuelve la clase del objeto.

- hashCode() → Devuelve un código de hash para el objeto this.
- toString() → Devuelve una representación en forma de string del objeto this.

{

- notify()
- notifyAll()
- wait()
- wait(long timeout)
- wait(long timeout, int mos)

 } Métodos relacionados con la ejecución multithreading (concurso)

HERENCIA

CLASE DE UN OBJETO

En Java se puede averiguar la clase de un objeto ejecutando el método `getClass()`.

También se puede averiguar si un objeto es instancia de una clase (directa o indirectamente), con el operador `instanceof`. Ex. `other instanceof Square`.

En Java, también se encierra el código propenso a producir excepciones en un bloque `try`, aunque en este caso no hay bloque `except` sino `catch`.

Ex. `catch (java.util.InputMismatchException err)`

TRATAR VARIAS EXCEPCIONES DEL MISMO

Para tratar varias excepciones diferentes de la misma manera se ponen en el mismo bloque `catch`, separadas por el operador `|`:

ACCESO A LA INFORMACIÓN DE LAS EXCEPCIONES

En Java se puede acceder a la información asociada a la excepción que se está tratando en un bloque `catch` usando el nombre dado al objeto que representa la excepción en la cabecera del bloque. Ex. `catch (ArithmeticException err)`

La clase base `Exception` ofrece varios métodos para obtener información diversa (`getCause`, `getLocalizedMessage`, `getMessage`, `getStackTrace`). Podría haber métodos adicionales.

CONTROL GENÉRICO DE EXCEPCIONES

En Java se puede poner un último bloque `catch` para recoger cualquier excepción no contemplada explícitamente en los bloques `catch` anteriores. Para ello se pone `Exception`. Ex. `catch (Exception err)`

BLOQUES ADICIONALES

En Java no existe equivalente al bloque `else` de Python. Si se puede añadir un bloque `finally` con el mismo objetivo que en Python.

LANTAMIENTO DE EXCEPCIONES

En Java podemos lanzar una excepción con la sentencia `throw`.

Ex

`throw new IllegalArgumentException("la lista no puede estar vacía")`

Tanto en Java como en Python se puede re- Lanzar la excepción recibida en un bloque de control usando `throw` o `raise` sin especificar ninguna excepción.

• DEFINICIÓN DE NUEVAS CLASES DE EXCEPCIONES

En Java se definen nuevas **clases de excepciones** creando una clase que herede, directa o indirectamente de la clase base Exception.

Ex.

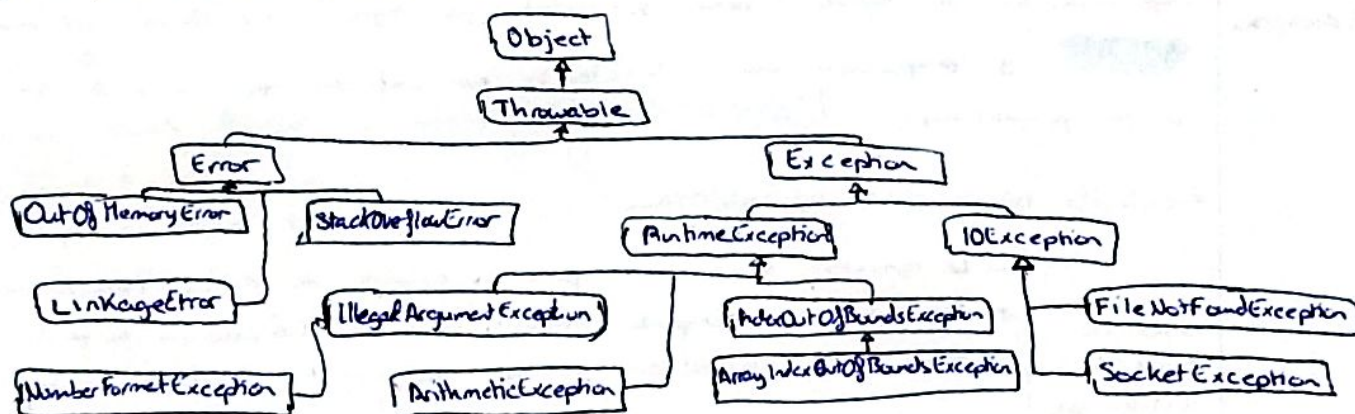
```
class MyException extends Exception {
}
```

• DECLARACION DE EXCEPCIONES SALIENTES

En Java, cuando un método **deja salir una excepción de cualquier clase** que no sea heredera, directa o indirecta, de la clase **RuntimeException**, debe **declararla** en su cabecera usando una cláusula **throws**.

```
public static double average (int[] listOfInt) throws MyException {
    if (listOfInt.length == 0) {
        throw new MyException(); // MyException hereda de Exception
    } else {
        return Arrays.stream (listOfInt).sum() / listOfInt.length;
    }
}
```

• JERARQUÍA DE EXCEPCIONES EN JAVA



En Java una clase abstracta es una clase marcada con el modificador **abstract**. Esta puede incluir métodos abstractos (métodos declarados pero no implementados) marcados con el modificador abstract.

Ex.

```
public abstract class Shape {
    public abstract double area();
}
```

Nota: no se pueden crear objetos de una clase abstracta. Solo sirven para crear subclases que implementen los métodos abstractos. Si no implementa todos los métodos de la subclase también será una clase abstracta.

• INTERFACES

En Java una interfaz es algo parecido a una clase completamente abstracta que se usa para proporcionar un listado de declaraciones de métodos (signaturas) que las clases que las clases que quieren cumplir la interfaz deben implementar.

```

Cr.
public interface IShape {
    public double area();
    public double perimeter();
}

```

Nota: Todos los métodos declarados en una interfaz son públicos, por lo que el modificador public podría omitirse.

Una clase que quiera adherirse al cumplimiento de una interfaz, lo indica con la palabra **implements** e implementa todos los métodos definidos en la interfaz.

```

En
public class Square implements IShape {
    double side;

    Square(double side) {
        this.side = side;
    }

    public double area() {
        return side * side;
    }

    public double perimeter() {
        return side * 4;
    }
}

```

Nota: Square no hereda de IShape, implementa lo susodicho. Java no permite la herencia múltiple, pero una clase puede heredar de otra e implementar varias interfaces al mismo tiempo.

CLASES

ABSTRACTAS

Aparte, una interfaz puede incluir declaraciones de constantes, tipos anidados, y la implementación de métodos static y métodos 'por defecto', que llevan el modificador **default** y proporciona una implementación por defecto en caso de que la clase no la proporcione.

CLASES ABSTRACTAS VS INTERFACES

USO DE
CLASES
ABSTRACTAS

→ La queremos como base para un conjunto de clases estrechamente relacionadas que posiblemente compartan algunos atributos (variables que se pueda modificar y/o métodos no abstractos) que estarían implementados en dicha clase base.

→ Queremos que algunos de los métodos que deben implementar las clases derivadas, puedan llevar los modificadores protected o private (los métodos declarados en una interfaz deben ser public).

USO DE
INTERFACES

→ Queremos definir un comportamiento que puede ser implementado por diferentes clases no relacionadas.

→ Queremos disponer de una aproximación a la herencia múltiple, posiblemente heredando de una clase e implementando varias interfaces.

CLONEABLE

En Java una clase implementa la interfaz **Cloneable** por indicarle al método `Object.clone()` que es legal que ese método haga una copia campo por campo de las instancias de esa clase.

Al invocar el método de clonación de `Object` de una instancia que no implementa la interfaz `Cloneable`, se genera la excepción **CloneNotSupportedException**.

Las clases que implementan la interfaz deben sustituir el método protegido `Object.clone()` con un método público

Ex:

```
public class CloneableClass implements Cloneable {
```

...

@Override

```
public CloneableClass clone() throws CloneNotSupportedException {
```

```
    CloneableClass newObject = (CloneableClass) super.clone();
```

```
    if (data != null) {
```

```
        newObject.data = this.data.clone(); // Hace una copia en profundidad.
```

```
    }
```

...

CLASES
ABSTRACTAS

El objeto clonado debe ser igual al objeto original cuando se comparan usando el método `equals()`, por lo que también será necesario redefinirlo (como hacemos en Java en el @tut4ordering).

• COMPARABLE <T>

Para definir cómo comparar los objetos de una clase, definiendo un orden total, esta debe implementar la interfaz `Comparable<T>`, donde T representa el tipo de la clase en cuestión.

Ex:

```
public class Pareja implements Comparable<Pareja> {
```

```
    int a, b;
```

```
    public Pareja (int a, int b) {
```

```
        this.a = a;
```

```
        this.b = b;
```

```
    }
```

```
    public int compareTo (Pareja o) {
```

```
        if (a > o.a) return 1;
```

```
        if (a < o.a) return -1;
```

```
        if (b > o.b) return 1;
```

```
        if (b < o.b) return -1;
```

```
        return 0;
```

```
    }
```

```
}
```

• Iterator <T> e Iterable <T>

En Java una clase es iterable si implementa la interfaz `java.lang.Iterable`, que obliga a definir un método llamado `iterator()`, que devuelve un iterador sobre un objeto de esa clase. Un iterador es un objeto de una clase que implementa la interfaz `java.util.Iterator`, que obliga a definir 2 métodos: `next()`, que devuelve uno por uno, los elementos de una secuencia obtenida a partir del objeto sobre el que itera, y el método `hasNext()`, que indica si quedan, o no, elementos por devolver. La clase `RangeIterator` es iterable, es su propio iterador.

```
import java.util.Iterator;
import java.lang.Iterable;
```

```
public class RangeIterator implements Iterator<Integer>, Iterable<Integer> {
    private int start = 0;
    " " step = 0;
    " " stop = 0;
    " " current = 0;

    public RangeIterator (int start, int step, int stop) {
        this.start = start;
        this.step = step;
        this.stop = stop;
    }

    public Iterator<Integer> iterator () {
        this.current = this.start;
        return this;
    }

    public boolean hasNext () {
        int result = this.current;
        this.current += this.step;
        return result;
    }
}
```

CLASES

ABSTRACTOS

• USO DE LOS ITERADORES

Un iterador se puede invocar usando sus métodos o a través de bucles for, que los invoca automáticamente.

```
import java.util.Iterator;
public class Main {
    public static void main (String[] args) {
        RangeIterator range = new RangeIterator (1, 2, 13);
        Iterator<Integer> iterator = range.iterator ();

        while (iterator.hasNext ()) {
            System.out.println (iterator.next ());
        }
    }
}
```

```
public class Main {
    public static void main (String[] args) {
        for (int e : new RangeIterator (1, 2, 13)) {
            System.out.println (e);
        }
    }
}
```

• OBJETO ITERABLE VS OBJETO ITERADOR

Un objeto Iterable puede ser su propio iterador, pero es + usual que el objeto iterable y el iterador que lo recorre sean objetos diferentes.

```
import java.util.Iterator;
import java.util.Iterable;
```

```
public class Range implements Iterable<Integer> {
    class RangeIterator implements Iterator<Integer> {
        private Range range;
        private int current;

        public RangeIterator (Range range) {
            this.range = range;
            this.current = range.start;
        }

        public boolean hasNext () {
            return this.current < this.range.stop;
        }
    }
}
```

```
* private int start = 0;
private int stop = 0;
private int step = 0;
```

```
public Range (int start, int step, int stop) {
    this.start = start;
    this.step = step;
    this.stop = stop;
}

public Iterator<Integer> iterator () {
    return new RangeIterator ();
}
}
```